

Learning with AI

Topic 2: Mobile App Development with React/PWA

Mausam Shrestha

CSC 494 — IoT | Spring 2026

Northern Kentucky University

Software Domain — How I built a real-time IoT dashboard accessible from any device

What Problem Did I Need to Solve?

My Smart Home Health Monitor generates sensor data 24/7.

People need to **see that data anywhere, from any device** — without installing an app.

Requirements:

- Real-time updates — data pushed every 10 seconds
- Historical charts (1 hour, 24 hours, 7 days, All Data)
- Works on phone AND desktop
- No App Store required
- LED/purifier control status
- Sleep quality logging interface

Solution: React + Vite + PWA + Firebase Realtime Database

What I Already Knew — And What Was New

My background: Several MERN stack projects (MongoDB, Express, React, Node.js)

Already knew	New for this project
React components, props, state	Vite as a build tool
<code>useEffect</code> , <code>useMemo</code> , hooks	Firebase Realtime Database
REST API calls from React	Real-time push with <code>onValue</code>
Deploying Node/Express backends	Firebase Hosting (no backend server)
MongoDB for data storage	Time-series data in RTDB

The challenge wasn't React itself — it was replacing the entire MERN backend with Firebase.

Why Vite Instead of Create React App?

What AI Helped Me Understand

In my previous MERN projects I used Create React App (CRA). For this project I switched to Vite.

AI explained the key differences:

	Create React App	Vite
Dev server startup	10–30 seconds	< 1 second
Hot reload	Rebuilds full bundle	Only sends changed module
Build output	Slower webpack bundle	Rollup — faster, smaller
Maintenance	Deprecated by Meta	Actively maintained

Replacing the MERN Backend with Firebase

The Biggest Mental Shift

In MERN stack: React calls **my own Express API** → Express queries **MongoDB** → returns JSON.

For this project: React talks **directly to Firebase** — no Express server, no Node.js, no backend I manage.

MERN approach:

React → `fetch('/api/readings')` → Express → MongoDB → JSON response

Firebase approach:

React → `onValue(ref(db, 'sensor_readings'))` → Firebase pushes data instantly

What I had to unlearn: Thinking in request/response. Firebase doesn't respond to requests — it *pushes* changes the moment they happen. The ESP32 writes to Firebase

Firestore Realtime Database Integration

Observer Pattern vs REST

Firestore Realtime Database with `onValue`: You *subscribe* → Firestore pushes every change to you instantly.

```
import { ref, query, limitToLast, onValue } from "firebase/database";

// Runs ONCE – callback fires every time data changes in Firestore
const q = query(ref(db, "sensor_readings"), limitToLast(500));
onValue(q, (snapshot) => {
  const data = snapshot.val() ?? {};
  const readings = Object.values(data);
  setReadings(readings); // React state updates → UI re-renders
});
```

AI insight: This is the **Observer Pattern** — the opposite of REST polling. In MERN I fetched data when I needed it. Here Firestore tells me when data changes. For a live

Why Realtime Database — Not Firestore

What AI Got Wrong — And What I Learned From It

AI initially recommended Firestore for my IoT dashboard.

Why that was wrong for my use case:

- Firestore charges per document read — with 10-second sensor pushes, costs add up fast
- Firestore is optimized for structured documents, not time-series streams
- Firebase Realtime Database is designed for exactly this: continuous numeric data, low-latency push, free tier covers IoT projects

How I found the right answer: Read Firebase's own comparison guide and looked at how other IoT projects structure their data.

Data Visualization with Recharts

Turning Numbers Into Understanding

Raw Firebase Realtime Database data:

```
{ "timestamp": 1710000000, "co2": 1247, "pm25": 18.3, "voc": 340 }
```

Recharts transforms it into interactive time-series charts:

```
<AreaChart data={readings}>
  <XAxis dataKey="timestamp" tickFormatter={formatTime} />
  <YAxis domain={[300, 2000]} />
  <Area dataKey="co2" stroke="#38bdf8" fill="#e0f2fe" dot={false} />
  <ReferenceLine y={1000} stroke="red" label="Danger" />
  <Tooltip />
</AreaChart>
```

Progressive Web App (PWA)

Making a Website Feel Like a Native App — Without Writing Native Code

What makes a PWA?

1. Web App Manifest (`manifest.json`)

- App name, icon, theme color
- Tells the browser "this can be installed to the home screen"

2. Service Worker (JavaScript file running in the background)

- Caches static assets for fast loading
- Enables offline support

PWA — What I Actually Learned

Adding One File to My Existing React App Made It a Mobile App

Before: Firebase-hosted React dashboard — desktop browser only.

After: With `manifest.json` linked in `index.html`, any phone user can:

- Open the site in Safari or Chrome
- Tap "Add to Home Screen"
- Launch it like any other app — full screen, no browser UI, my icon

No App Store. No developer account. No Xcode. No additional code.

AI helped me understand: `display: standalone` removes the browser chrome, making it feel native. This is the same mechanism used by Twitter Lite, Pinterest, and Starbucks' mobile ordering site.

Firestore Hosting — Replacing Express

Deploying Without a Server

In MERN projects, I deployed a Node/Express server (Heroku, Render, etc.) to serve the API and static files.

For this project, Firestore Hosting serves the React build directly — no server to manage, no process to keep alive, no port to expose.

```
npm run build          # Vite builds to /dist
firebase deploy        # Uploads /dist to Firestore CDN globally
```

Result: `smart-home-health-monitor.web.app` — live on a global CDN, HTTPS by default, zero server maintenance.

Key learning: Serverless hosting fundamentally changes the deployment model.

Results — What I Built and Deployed

Dashboard features built and working:

- ✓ Real-time sensor cards — CO₂, PM_{2.5}, VOC, Temperature, Humidity
- ✓ Time-series charts for all 5 sensors (1h / 24h / 7d / All Data)
- ✓ Threshold alert badges (CO₂ 1000 ppm, PM_{2.5} 35 µg/m³, VOC 2500)
- ✓ LED/purifier step chart — shows exact ON/OFF timing over selected range
- ✓ Sensor health detection — auto-flags stuck or disconnected sensors
- ✓ Sleep quality rating interface (1–10 input, stored in Firebase)
- ✓ PWA installable on iOS Safari and Android Chrome
- ✓ Deployed on Firebase Hosting — publicly accessible without login

Key Takeaway

Knowing React was not enough. The new challenge was replacing the entire backend mental model — from request/response REST APIs to a push-based real-time database, and from server-managed deployments to serverless hosting. That shift in thinking, not the React code itself, was the real learning.

AI accelerated the Firebase and Vite learning significantly — but understanding *why* each architectural choice was made required reading documentation and building the actual system.

Repository: <https://github.com/shresthamausam07/smart-home-health-monitor>

Learning docs: <https://github.com/shresthamausam07/mausam-shrestha-learning-with-ai>

Thank You

Mausam Shrestha

CSC 494 — IoT | Spring 2026

shrestham2@mymail.nku.edu

Topic 2 complete — Mobile App Development with React/PWA